# Agent Self-Evolution

Phase 3 Validation Report
System-prompt section evolution via splice-and-restore

---

Date: June 25, 2026
Repository: github.com/jramos/agent-self-evolution

## Executive Summary

Agent Self-Evolution is a standalone optimization pipeline that uses DSPy and GEPA (Genetic-Pareto Prompt Evolution) to automatically improve an agent's skills, tool descriptions, system prompts, and code through evolutionary search — all via API calls with no GPU training required. Phase 1 shipped a synthetic-only deploy gate; Phase 2 made it behavior-aware and brought tool-description parity. Phase 3 extends the framework to the third instructions surface — *named sections of the agent's system prompt* — evaluated end-to-end against the real agent.

This report documents the Phase 3 validation of system-prompt section evolution. The target is a top-level string constant in Hermes Agent's `prompt_builder.py` (here, **MEMORY_GUIDANCE**), evolved via GEPA and validated *purely behaviorally*: every candidate is spliced into the live prompt file and scored by running the real agent (`hermes -z`) against a curated task suite — there is no synthetic LLM-as-judge signal to lean on. Production **MEMORY_GUIDANCE** is already well-tuned, so the saturation pre-flight correctly default-denies it (no headroom). To exercise the loop end-to-end, the headline run evolves a *deliberately-weakened* baseline (supplied via `--baseline-override-file`): the agent's holdout pass-rate moved **67%** → **100%** (4/6 → 6/6 tasks, **+2W / 0L**) while the section shrank **-15.2%**. The closed-loop deploy gate decided **DEPLOYED**, and the live prompt file was restored byte-for-byte after every trial.

| KEY RESULT — MEMORY_GUIDANCE (prompt-section deploy via closed-loop gate) |
|---|
| **Holdout pass-rate (n=6): 67% → 100% (+2W / 0L)** |
| **Tasks passing: 4/6 → 6/6** |
| **Section size: 330 → 280 chars (-15.2%)** |
| **Decision: DEPLOYED via the closed-loop behavioral gate** |

## Background

Agent Self-Evolution targets the instructions layer of an LLM agent — skill files, tool descriptions, and system prompts — and evolves the text via API-only evolutionary search. An agent's behavior is governed by three layers:

| Layer | What It Is | How It's Currently Improved |
|---|---|---|
| Model Weights | The underlying LLM (Claude, GPT, etc.) | RL training (Tinker-Atropos) |

| | | |
|---|---|---|
| Instructions | Skills, system prompts, tool descriptions | Manual authoring (static) |
| Tool Code | Python implementations of each tool | Manual development |

Phases 1 and 2 validated skill files and tool descriptions. Phase 3 completes the instructions trio with **system-prompt sections** — the highest-leverage, widest blast-radius surface, since one section governs the agent across every task. The section is a string constant inside Hermes' own source, so unlike the skill path (separate writable workdir) there is no env-var hook or plugin seam: the framework edits `prompt_builder.py` in place. The integration is an AST-precise **splice-and-restore** — the candidate is byte-spliced into the live file for the duration of a trial and restored from an atomic backup afterward (`flock` + checksum-drift detection + parse-guard, reused from the Phase 2 closed-loop validator). Crucially, a system-prompt section has no cheap synthetic proxy: the only honest measure of "did this guidance help" is running the real agent, so Phase 3's deploy gate is purely behavioral.

# Approach: Behavioral Prompt-Section Evolution

## Three Optimization Engines

| Engine | What It Optimizes | License | Role |
|---|---|---|---|
| DSPy + GEPA | Skills, prompts, tool descriptions | MIT | Primary (validated) |
| DSPy MIPROv2 | Few-shot examples, instruction text | MIT | Fallback optimizer |
| Iterative test-feedback repair | Tool implementation code | MIT | Code repair (Phase 4) |

**GEPA** (Genetic-Pareto Prompt Evolution) is the star engine — an ICLR 2026 Oral paper from Stanford/UC Berkeley. Unlike traditional evolutionary search that only sees pass/fail scores, GEPA reads full execution traces to understand *why* things failed, then proposes targeted mutations. Phase 3 wires GEPA to a sentinel-preserving proposer (mutations are confined to the section's text, never the surrounding scaffolding) and routes every candidate score through a real `hermes -z` subprocess. Because the spliced `prompt_builder.py` is a single shared file and DSPy evaluates with a thread pool, candidate scoring is serialized under a lock — an accepted cost of the splice-and-restore model.

## The Optimization Pipeline

1. **Resolve baseline** — Read the section's current text from `prompt_builder.py` (or accept a weakened baseline via `--baseline-override-file` to create headroom on an already-tuned section)
2. **Split** — Deterministic seeded train / holdout split of the curated JSONL task suite

3. **Saturation pre-flight** — Score the baseline behaviorally on the holdout; a `no_headroom` band default-denies (correctly refusing to evolve a saturated section) unless overridden

4. **GEPA loop** — The section text is a sentinel-delimited region of a passthrough predictor's instructions; GEPA mutates it with the sentinel-preserving proposer. Each candidate is spliced into the live file and scored by running the agent on each task

5. **Compound verdict** — Layer 1: did the agent invoke the expected tool (e.g. `memory`)? Layer 2: an LLM judge scores the saved content against each task's rubric

6. **Closed-loop deploy gate** — Select the GEPA val-best candidate, then run baseline vs. evolved on the holdout suite; deploy iff holdout pass-rate doesn't regress and per-task wins offset losses ≥ 2:1

7. **Report + restore** — Structured `gate_decision.json` (v5 schema, prompt-section variant); the live file is restored byte-for-byte

The honest Phase 3 story is two-part. First, the framework's **regression-catching discipline**: the production `MEMORY_GUIDANCE` is already well-tuned, so a capable agent satisfies the suite regardless of small wording changes — the saturation pre-flight scores the baseline at ceiling and correctly *default-denies*, refusing to spend GEPA budget where no improvement is possible. This mirrors the Phase 2 finding that the framework is improvement-finding only where headroom genuinely exists. Second, to demonstrate that the loop produces a real, grounded improvement when headroom *does* exist, the headline run evolves a deliberately-adversarial baseline (one that instructs the agent *not* to save) — exactly the weakened-target approach Phase 2 used for its headline. That run consumed **$0.03** across 46 in-process LM calls in ~7 minutes (the agent's own subprocess spend is separate). Splicing a different section measurably changed live agent behavior, and GEPA recovered a corrected section that the closed-loop gate deployed.

# Phase 3 Experiment

## Configuration

| Parameter | Value |
|---|---|
| Target Section | MEMORY_GUIDANCE — evolved from a deliberately-weakened baseline (production MEMORY_GUIDANCE is saturated; the weak baseline, supplied via --baseline-override-file, exercises the loop end-to-end) |
| Baseline Size | 330 characters |
| Optimizer LM | openai/gpt-5.4-mini |
| Reflection LM (GEPA) | openai/gpt-5.4-mini |
| Content-Judge LM (Layer 2) | openai/gpt-5.4-mini |
| Agent (hermes -z) | openai/gpt-5.4-mini (Hermes-configured default) |

| | |
|---|---|
| Behavioral Suite | 6 holdout tasks (real hermes -z, scored end-to-end) |
| Total Optimization Time | 449 seconds (~7 minutes) |
| Total LM Calls (in-process) | 46 |
| Total Cost (USD, in-process) | $0.03 |
| Deploy Gate | closed-loop behavioral — holdout pass-rate no-regression + per-task wins ≥ 2·losses; compound verdict (Layer 1 trigger + Layer 2 content judge) |
| Saturation Pre-flight | forced via --force-saturation-check (the weakened baseline had real headroom; production MEMORY_GUIDANCE default-denies as no_headroom) |

### Evaluation Suite

The evaluation suite is a curated, hand-authored JSONL benchmark (`memory_guidance.jsonl`, 12 tasks across five categories: save-preference, save-correction, dont-save-task-progress, dont-save-completed-work-log, and declarative-vs-imperative). Unlike Phases 1 and 2, there is *no* synthetically-generated train/val/holdout of LLM-judge examples — every task is scored behaviorally by running the real agent, and the deploy gate's holdout is 6 of those tasks. Each save task carries an `expected_save_content` rubric consumed by the Layer 2 content judge.

### Fitness Function

Fitness is behavioral, not a synthetic judge score. For each task, the candidate section is spliced into the live `prompt_builder.py`, the agent runs once via `hermes -z`, and the resulting session is read back from Hermes' SQLite session store. The verdict is compound:

```
pass = Layer1(expected memory action fired, forbidden actions absent) AND
       Layer2(content-judge score ≥ 0.7 on save tasks)
```

GEPA's reflection LM reads the per-task failures and proposes a targeted mutation of the section text; the sentinel-preserving proposer confines edits to the section and re-raises rather than admit a candidate that drops the markers. The deploy gate then re-runs baseline vs. evolved on the holdout and decides on the behavioral signal alone — holdout pass-rate no-regression plus a per-task win/loss rule — with no paired-bootstrap CI, because there is no synthetic per-example distribution to resample.

## Results

| Metric | Baseline | Evolved | Δ |
|---|---|---|---|
| **Section size (chars)** | 330 | **280** | -15.2% |
| **Holdout pass-rate (n=6)** | 67% | 100% | +33% |

| | | | |
|---|---|---|---|
| **Tasks passing (n=6)** | 4/6 | 6/6 | +2W / 0L |
| **Decision** | — | DEPLOYED | **via closed-loop** |

Evolving the weakened **MEMORY_GUIDANCE** baseline, the agent's holdout pass-rate moved **67% → 100%** (4/6 → 6/6 tasks, **+2 wins / 0 losses**, 4 ties) while the section text *shrank* **-15.2%** (330 → 280 chars). GEPA learned from the save-task failures and inverted the adversarial instruction — it removed the "never proactively save" misdirection and restored proactive saving while keeping the legitimate "don't store passing remarks" discrimination, in fewer characters. **Decision: DEPLOYED** via the closed-loop gate (evolved pass_rate 1.00 >= baseline 0.67; per-task: 2 wins, 0 losses, 4 ties). The proposer rejected 0 sentinel-breaking candidates. Throughout, the live `prompt_builder.py` was restored byte-for-byte after every trial. The production MEMORY_GUIDANCE itself is saturated and correctly default-denies — the framework is regression-catching, and only finds improvements where real headroom exists.

### How the Result Was Produced

GEPA evolves the section text through a reflective loop; the gate then reads the behavioral signal:

1. Splice a candidate section into the live `prompt_builder.py` (only when the candidate changes); run each holdout task once via `hermes -z` and read the session from Hermes' `state.db`
2. Score each run with the compound verdict (Layer 1 tool-trigger membership + Layer 2 content judge on memory-save content); abstentions (agent/runner errors) score 0 in-loop and tie at the gate
3. The reflection LM reads the failures and proposes a sentinel-confined mutation of the section text; GEPA accepts on improvement-or-equal
4. Select the GEPA val-best candidate; run the closed-loop deploy gate (baseline vs. evolved on 6 holdout tasks, its own backup/restore)
5. **Decide** — Deploy iff evolved holdout pass-rate ≥ baseline AND per-task wins offset losses ≥ 2:1. On this run: 67% → 100%, 2W/0L → DEPLOY

Two design choices made this outcome trustworthy. First, the splice-and-restore guard (atomic backup + exclusive `flock` + byte-restore, with stale-backup refusal) means the user's Hermes checkout is never left mutated, even on crash. Second, the deploy gate is the same proven closed-loop validator used for tool descriptions — the prompt path adds only a thin installer plus a per-task content judge, so the decision rule, audit trail, and restore machinery are shared and already battle-tested. The behavioral-only design is not a shortcut: it is the only honest measure for a system-prompt section, which has no cheap synthetic proxy.

## Safety and Guardrails

Every evolved section must clear these constraints, and the live prompt file is protected throughout:

| Constraint | Enforcement | Status |
|---|---|---|
| Self-evolution test suite | 1,232 pytest tests pass on the optimizer itself | Implemented |
| Byte-clean splice/restore | Atomic backup + byte-for-byte restore of prompt_builder.py after every run | Implemented |
| Parse-guarded write | Candidate spliced via repr() + ast.parse check; refuses to write non-parseable Python | Implemented |
| Exclusive lock + drift check | flock on the prompt file's dir + sha-drift detection; stale-backup refusal on startup | Implemented |
| Compound verdict | Layer 1 tool-trigger membership AND Layer 2 LLM content judge (≥ threshold) | Implemented |
| Abstain on corrupt session | A malformed agent session abstains (neutral), never scores as a behavioral regression | Implemented |
| Closed-loop deploy gate | Holdout pass-rate no-regression + per-task wins ≥ 2·losses | Implemented |
| Saturation pre-flight | Default-denies a saturated (no_headroom) section before spending GEPA budget | Implemented |
| Budget ceiling | --max-cost-usd aborts on in-process LM spend overrun | Implemented |
| Deployment via apply + review | --apply writes the section; PR automation deferred for prompt sections | By design |
| Benchmark regression | External --benchmark-cmd hook (TBLite / harness) | Planned |

The source Hermes repository is never left modified: the section is spliced in only for the duration of a trial and restored from an atomic backup, and all evolution output (gate decisions, section before/after text, run logs) is written under the framework's local `output/` directory. PR automation is deferred for prompt sections — a section-scoped PR path is future work — so the deploy step is an explicit `--apply` plus a human-authored pull request.

## Roadmap

| Phase | Target | Engine | Timeline | Status |
|---|---|---|---|---|
| Phase 1 | Skill files (SKILL.md) | DSPy + GEPA | 3-4 weeks | Validated ✓ |
| Phase 2 | Tool descriptions | DSPy + GEPA | 2-3 weeks | Validated ✓ |
| Phase 3 | System prompt sections | DSPy + GEPA | 2-3 weeks | Validated ✓ |

| | | | | |
|---|---|---|---|---|
| Phase 4 | Tool implementation code | Iterative test-feedback repair | 3-4 weeks | Validated ✓ |
| Phase 5 | Continuous improvement | Propose-only triage sentinel | 2 weeks | Shipped ✓ |

Phase 3 completes the instructions trio — skills, tool descriptions, and now system-prompt sections — all gated by the same closed-loop discipline. The behavioral-only deploy gate proves the framework can evolve the highest-blast-radius instructions surface safely: it default-denies a saturated section, produces a real grounded improvement where headroom exists, and never leaves the agent's source mutated. Phase 4 (tool implementation code) and Phase 5 (continuous improvement) extend the framework beyond the instructions layer.

## Immediate Next Steps

1. **Harder behavioral suites** — Production system-prompt sections are heavily tuned and saturate the current suites; develop richer, harder task suites (and weaker agent tiers) so headroom exists on real targets, not only adversarial baselines.
2. **Additional sections** — The same path supports any string-constant section (SKILLS_GUIDANCE, SESSION_SEARCH_GUIDANCE, etc.); MEMORY_GUIDANCE was the first proof point, chosen for its clear tool-call anchor.
3. **Section-scoped PR automation** — Wire --create-pr for prompt sections by splicing into origin/<base>'s prompt_builder.py (not the local checkout), so the PR diff carries only the section change.
4. **Agent-side cost capture** — The agent's own LM spend happens inside the hermes subprocess and is invisible to the in-process budget ceiling; surface it from the session store so --max-cost-usd accounts for end-to-end spend.